

MODULE 2.1: UNSUPERVISED LEARNING

Contents

Unsupervised Learning	1
Clustering	2
K-Means Clustering.....	4
Kernel k-Means Clustering.....	6
Principal Component Analysis (PCA)	6
Hierarchical Clustering	10

Unsupervised Learning

Unsupervised learning is a type of machine learning where the model is trained on data without labelled outcomes. The goal is to discover hidden patterns or intrinsic structures within the input data. Unlike supervised learning, there is no predefined output, and the model learns from the raw data itself. **Key**

Concepts:

- **No Labels:** Data used in unsupervised learning does not have associated target outputs or labels.
- **Data Exploration:** The focus is on exploring the data and finding relationships, clusters, or patterns.
- **Dimensionality Reduction:** Techniques used to reduce the number of features while retaining essential information.

Applications of Unsupervised Learning

1. **Market Segmentation:** Grouping customers based on purchasing behaviour to tailor marketing strategies.
2. **Anomaly Detection:** Identifying fraudulent transactions or network intrusions by detecting outliers in data.
3. **Recommendation Systems:** Suggesting products or content to users based on similar preferences found in data.
4. **Customer Profiling:** Understanding customer behaviour patterns to improve user experience.
5. **Image Compression:** Reducing the size of images while retaining important features using dimensionality reduction techniques.
6. **Document Clustering:** Grouping similar documents together for information retrieval or topic modelling.

Challenges in Unsupervised Learning

1. **Evaluation:** Lack of ground truth makes it difficult to evaluate the quality of results.
2. **Interpretability:** Understanding and interpreting the patterns or clusters found can be complex.
3. **Scalability:** Some algorithms may struggle to scale effectively with large datasets.

4. **Choosing the Right Algorithm:** Selecting the appropriate technique for a specific task can be challenging without labelled data.
5. **Data Quality:** The effectiveness of unsupervised learning heavily depends on the quality and preprocessing of the input data.

Clustering

Clustering is a type of unsupervised learning that involves grouping a set of objects (data points) into clusters based on their similarities. The objective is to ensure that objects within the same cluster are more like each other than to those in other clusters. Clustering helps in identifying natural groupings in data, making it easier to analyse and interpret. **Key Concepts:**

- **Distance Metrics:** Measures used to quantify the similarity or dissimilarity between data points. Common metrics include Euclidean, Manhattan distance, and cosine similarity.
 - **Centroids:** The central point of a cluster, often used in algorithms like K-Means to represent the cluster.
 - **Inertia:** A measure of how tightly the clusters are packed together (to assess the quality of the clustering).
1. **K-Means Clustering:** Partitions the data into K clusters by assigning each data point to the nearest centroid and updating the centroids based on the mean of the points in each cluster. **Use Cases:** Customer segmentation, image compression. **Limitations:** Requires the number of clusters (K) to be specified in advance and is sensitive to outliers.
 2. **Hierarchical Clustering:** Builds a hierarchy of clusters using either a bottom-up approach (agglomerative) or a top-down approach (divisive). The result is often represented in a dendrogram. **Use Cases:** Taxonomy classification, social network analysis. **Limitations:** Computationally expensive for large datasets.

Aspect	Supervised Learning	Unsupervised Learning
Labeled Data	Requires labeled data (input-output pairs)	Works with unlabeled data (only input)
Goal	Predict or classify outcomes based on past data	Explore the data and find hidden structures or patterns
Output	Produces known outputs like categories or numeric values	No predefined output; identifies inherent structure
Type of Problems	Classification and Regression	Clustering and Association
Training Process	Learns mapping from inputs to outputs	Learns patterns or grouping within input data
Evaluation	Easily evaluated with metrics like accuracy, precision, recall, MSE	Harder to evaluate; often uses visual inspection or silhouette score
Algorithms	Decision Trees, k-NN, SVM, Linear Regression, Neural Networks	K-Means, DBSCAN, Hierarchical Clustering, PCA, Autoencoders
Data Dependency	Performance depends highly on quantity and quality of labeled data	Can work with unlabeled data, but quality still matters

Complexity	Generally more computational due to labeled data processing	Often less complex initially but may require deeper analysis
Examples	Email spam detection, credit scoring, disease prediction	Market segmentation, document clustering, image compression
Accuracy	Can be very accurate with good training data	Accuracy is relative; effectiveness depends on the clarity of discovered structure
Feature Engineering	Features are often engineered based on known outputs	Feature discovery can be automatic or exploratory
Usage Stage	Often used in deployment or production systems (e.g., ML models in apps)	Often used in research, exploration, or pre-processing
Learning Feedback	Learns with feedback (error correction)	No direct feedback during training
Dependence on Output	Strongly depends on the existence of correct output labels	Focuses only on input patterns

Aspect	Classification	Regression	Clustering
Learning Type	Supervised	Supervised	Unsupervised
Output Type	Discrete labels (class categories)	Continuous numeric values	Groups/clusters formed automatically
Prediction Type	Class membership	Value estimation	Pattern discovery
Goal	Separate data into predefined classes	Estimate numeric outcomes	Identify natural groupings in data
Examples	Spam detection, image recognition	House price prediction, demand forecasting	Customer segmentation, gene grouping
Common Algorithms	Logistic Regression, SVM, k-NN, Decision Trees	Linear Regression, Polynomial Regression, SVR	k-Means, DBSCAN, Agglomerative Clustering
Loss Function	Cross-entropy, hinge loss	MSE, MAE	Inertia, distance-based metrics
Evaluation Metrics	Accuracy, Precision, Recall, F1-score, ROC-AUC	MSE, RMSE, MAE, R^2	Silhouette score, Davies-Bouldin index
Nature of Output	Finite set of classes	Infinite real values	Cluster labels (not predefined)
Data Requirement	Requires labeled data	Requires labeled data	Works without labels
Decision Boundary	Creates class boundaries	Fits a curve or line	Partitions data space into groups
Use Cases	Classification problems	Prediction problems	Exploratory analysis

Interpretability	Often moderate	Often high (linear)	Lower; clusters may be ambiguous
Sensitivity to Outliers	Depends on algorithm	High (in many regressions)	High for k-means, low for DBSCAN
Type of Target Variable	Categorical	Numerical	None (latent structure)
Application Domains	NLP, vision, medical diagnosis	Finance, forecasting, engineering	Marketing, bioinformatics, anomaly detection
Underlying Task	Assign a label	Predict a quantity	Find structure in data

K-Means Clustering

- K-Means is a **partition-based** unsupervised clustering algorithm that divides a dataset into **K non-overlapping clusters**, where each data point belongs to the cluster with the **nearest mean** (centroid).
- **Objective:** Minimize the within-cluster sum of squared distances (or variance) between data points and their assigned cluster centroids.
- **K:** The number of clusters to be formed, which must be specified in advance
- **Cluster:** A group of similar data points.
- **Centroid:** The center (mean) of a cluster.
- **Distance metric:** typically, Euclidian distance

Objective Function (Loss function):

$$SSE = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

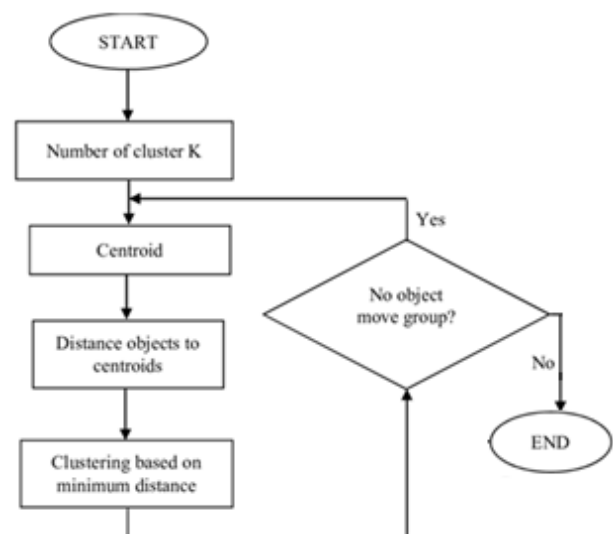
- k = number of clusters
- C_i = points in cluster i
- μ_i = centroid of cluster i

Advantages:

- Simple and fast to implement.
- Efficient for large datasets.
- Works well with **spherical, equally sized** clusters.

Disadvantages:

- Requires pre-specifying the number of clusters k .
- Sensitive to **initial centroid positions**.
- Not suitable for **non-convex** clusters.
- Struggles with **outliers** and **noisy data**.



Applications:

- Image segmentation
- Market/customer segmentation
- Document clustering
- Anomaly detection

Example

- For K=2 perform K means clustering algorithm. Assume A and C are the initial centroids.
- Calculate objective/Loss function of the centroid after they converge.

Point	X	Y
A	2	10
B	2	5
C	8	4
D	5	8
E	7	5
F	6	4

Centroids: A(2,10) & C(8,4)

Point	Distance from A	Distance from C	Ans
A(2,10)	0	8.4853	C ₁
B(2,5)	5	6.0828	C ₁
C(8,4)	8.4853	0	C ₂
D(5,8)	3.6056	5	C ₁
E(7,5)	7.0711	1.4142	C ₂
F(6,4)	8.9442	2	C ₂

∴ Cluster 1: A, B, D

Cluster 2: C, E, F

$$\text{New centroid} = \left(\frac{2+2+5}{3}, \frac{10+5+8}{3} \right) = (3, 7.6667)$$

(C₁)

$$\text{new centroid} = \left(\frac{8+7+6}{3}, \frac{4+5+4}{3} \right) = (7, 4.3333)$$

(C₂)

point	Dis from C ₁	Dis from C ₂	Ans.
A(2,10)	2.5383	5.5378	C ₁
B(2,5)	2.8480	5.0443	C ₁
C(8,4)	6.2004	1.0541	C ₂
D(5,8)	2.0278	4.1767	C ₁
E(7,5)	4.8074	0.6667	C ₂
F(6,4)	4.7376	1.0541	C ₂

Same C₁ & C₂ for both iteration

∴ Convergence.

$$\text{Loss function}(C_1) = \sum (x_i - C_1)^2$$

$$= (2.5383)^2 + (2.8480)^2 + (2.0278)^2 = 18.6652$$

$$\text{Loss func}(C_2) = (1.0541)^2 + (0.6667)^2 + (1.0541)^2 = 2.6667$$

$$\text{Total Loss function} = 18.6652 + 2.6667 = 21.3319$$

Kernel k-Means Clustering

- **Kernel k-Means** is an **extension of the standard k-Means clustering algorithm** that allows it to handle **non-linearly separable data** by using the **kernel trick** — just like in SVMs.
- Ordinary k-Means partitions data based on Euclidean distance in the input space, which works well only when clusters are roughly spherical and linearly separable. Kernel k-Means solves this limitation by performing clustering in a **high-dimensional feature space** where the data becomes linearly separable.
- Example: In image recognition, Kernel K-Means helps cluster images with complex variations not easily separable in the original space.

The main idea is to **map data points to a higher-dimensional feature space** using a transformation

$$\phi(x) : \mathbb{R}^n \rightarrow \mathbb{R}^m$$

and then perform standard k-Means there.

However, since computing $\phi(x)$ explicitly can be very costly, we use the **kernel trick**, where we only need the **dot product** between two mapped points:

$$K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$$

This allows clustering to be done **implicitly** in the higher-dimensional space.

Advantages

- Can detect **non-linear cluster boundaries**.
- No need to explicitly map data into high-dimensional space.
- Works well for data like images, text, or any dataset where linear k-Means fails.

Limitations

- Requires selecting an appropriate **kernel function** and parameters (like γ in RBF).
- Computationally more expensive (depends on N^2 kernel matrix).
- Sensitive to initialization, similar to regular k-Means.

Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a widely used statistical method for reducing the dimensionality of large datasets while retaining as much information (variance) as possible. It is an unsupervised technique that transforms the data into a new coordinate system where the axes (principal components) are ordered based on the variance they capture. PCA is especially useful in exploratory data analysis and for visualizing high-dimensional data. **Key Concepts in PCA:**

- **Principal Components (PCs):** The new axes in the transformed space. These components are the eigenvectors of the covariance matrix, and each successive component captures the remaining variance orthogonally.
- **Eigenvalues:** Measure the variance captured by each principal component. Larger eigenvalues correspond to principal components that capture more variance in the data.

- **Dimensionality Reduction:** PCA reduces the dimensionality of the data by selecting only the top k eigenvectors. This allows for a more compact representation of the data.
- **Variance Explained:** The ratio of the variance captured by a given principal component relative to the total variance. The first principal component captures the most variance, the second captures the second-most variance, and so on.

Applications of PCA:

- **Data Visualization:** PCA is often used to reduce the dimensionality of high-dimensional data for visualization. For example, reducing data to 2 or 3 dimensions allows us to plot complex data in a 2D or 3D scatter plot.
- **Noise Reduction:** By discarding components with small eigenvalues (which correspond to less variance), PCA can be used to remove noise from data, leading to better performance in machine learning models.
- **Feature Extraction:** PCA can extract features that summarize the data efficiently. The new features (principal components) are linear combinations of the original features and capture the most important patterns in the data.
- **Preprocessing for Machine Learning:** PCA is commonly used as a preprocessing step to reduce the dimensionality of data before feeding it into machine learning algorithms, particularly when dealing with high-dimensional data such as images or gene expression data.

Advantages of PCA:

- **Reduces Dimensionality:** PCA helps reduce the number of features in a dataset while retaining most of the information, which can improve the performance of machine learning models.
- **Improves Efficiency:** By reducing the number of features, PCA can make the computations more efficient and decrease the time required for training machine learning models.
- **Handles Correlation:** PCA can handle correlated features effectively by transforming the data into a new coordinate system where the features are uncorrelated.

Disadvantages of PCA:

- **Loss of Interpretability:** The principal components are linear combinations of the original features, so it can be difficult to interpret what the new features represent.
- **Sensitive to Scaling:** PCA is sensitive to the scale of the features, so it is essential to standardize the data before applying PCA if the features have different units or variances.
- **Linear Assumptions:** PCA assumes that the relationships between features are linear, which may not always be true for complex datasets.

EXAMPLE:

Find the Principal Component of each point (X_1, X_2) using PCA

F	Ex 1	Ex 2	Ex 3	Ex 4
X_1	4	8	13	7
X_2	11	4	5	14

Answer

$$\text{Mean } \bar{X}_1 = \frac{4+8+13+7}{4} = 8 \quad \bar{X}_2 = \frac{11+4+5+14}{4} = 8.5$$

Covariance Matrix

$$\text{Covariance Matrix (Cov)} = \begin{bmatrix} \text{Cov}(X, X) & \text{Cov}(X, Y) \\ \text{Cov}(Y, X) & \text{Cov}(Y, Y) \end{bmatrix}$$

$$\begin{aligned} \text{Cov}(X, X) &= \frac{1}{N-1} \sum (X - \bar{X})(X - \bar{X}) = \frac{1}{N-1} \sum (X - \bar{X})^2 \\ &= \frac{1}{3} ((4-8)^2 + (8-8)^2 + (13-8)^2 + (7-8)^2) \\ &= \frac{1}{3} (16 + 0 + 25 + 1) = 14 \end{aligned}$$

$$\begin{aligned} \text{Cov}(X, Y) &= \frac{1}{N-1} \sum (X - \bar{X})(Y - \bar{Y}) \\ &= \frac{1}{3} ((4-8)(11-8.5) + (8-8)(4-8.5) + (13-8)(5-8.5) + (7-8)(14-8.5)) \\ &= \frac{1}{3} (-4 \times 2.5 + 0 + 5(-3.5) + (-1)(5.5)) \\ &= \frac{1}{3} (-10 - 17.5 - 5.5) = -11 \end{aligned}$$

$$\text{Cov}(Y, X) = -11$$

$$\begin{aligned} \text{Cov}(Y, Y) &= \frac{1}{N-1} \sum (Y - \bar{Y})^2 = \frac{1}{3} ((11-8.5)^2 + (4-8.5)^2 + (5-8.5)^2 + (14-8.5)^2) \\ &= \frac{1}{3} (6.25 + 12.25 + 12.25 + 30.25) = 23 \end{aligned}$$

$$\therefore \text{Cov} = \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$

Eigen values

$$\begin{vmatrix} 14 - \lambda & -11 \\ -11 & 23 - \lambda \end{vmatrix} = 0$$

$$\Rightarrow 322 - 37\lambda + \lambda^2 - 121 = 0$$

$$\Rightarrow \lambda^2 - 37\lambda + 201 = 0$$

$$\lambda = \frac{37 \pm \sqrt{1369 - 804}}{2} = \frac{37 \pm 23.7697}{2}$$

$$= 30.3849, 6.6152$$

$$\text{Eigen value } \lambda_1 = 30.3849$$

$$\lambda_2 = 6.6152$$

Eigen vector

where,

$$\begin{bmatrix} 14-\lambda & -11 \\ -11 & 23-\lambda \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = 0 \quad U = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \text{ is eigen vector}$$

② We choose largest eigen value λ_1

$$\therefore \begin{bmatrix} 14 - 30.3849 & -11 \\ -11 & 23 - 30.3849 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = 0$$

$$\Rightarrow \begin{bmatrix} -16.3849 & -11 \\ -11 & 7.3849 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = 0$$

$$\Rightarrow \begin{bmatrix} -16.3849 u_1 - 11 u_2 \\ -11 u_1 + 7.3849 u_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\text{Let } -16.3849 u_1 = 11 u_2 = t$$

$$\Rightarrow \frac{u_1}{11} = \frac{u_2}{-16.3849} = t$$

$$\therefore U = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} 11 \\ -16.3849 \end{bmatrix}$$

$$\text{To find unit eigen vector, } |U| = \sqrt{11^2 + 16.3849^2} = 19.7349$$

$$\therefore \text{Unit eigen vector} = \frac{1}{19.7349} \begin{bmatrix} 11 \\ -16.3849 \end{bmatrix} = \begin{bmatrix} 0.5574 \\ -0.8302 \end{bmatrix}$$

(e)

Principal Component evaluation

X	Y	(X - $\bar{X}$)	(Y - $\bar{Y}$)
4	11	-4	2.5
8	4	0	-4.5
13	5	5	-3.5
7	14	-1	5.5

$\therefore$ To find Principal component,

$$e^T (x - \bar{x})$$

$$\therefore y_1 = [0.5574 \quad -0.8302] \begin{bmatrix} -4 \\ 2.5 \end{bmatrix} = -4.3051$$

$$y_2 = [0.5574 \quad -0.8302] \begin{bmatrix} 0 \\ -4.5 \end{bmatrix} = 3.7359$$

$$y_3 = [0.5574 \quad -0.8302] \begin{bmatrix} 5 \\ -3.5 \end{bmatrix} = 5.6927$$

$$y_4 = [0.5574 \quad -0.8302] \begin{bmatrix} -1 \\ 5.5 \end{bmatrix} = -5.1235$$

Hierarchical Clustering

- Hierarchical Clustering is an **unsupervised learning algorithm** that builds a hierarchy of clusters either from the **bottom-up** (agglomerative) or **top-down** (divisive).
- It does not require the number of clusters to be defined in advance.
- It groups data points together that are close to each other based on the measure of distance.

Type	Description
Agglomerative (Bottom-Up)	Start with each data point as its own cluster. Iteratively merge the closest clusters until one cluster remains or a stopping criterion is met.
Divisive (Top-Down)	Start with all data in one cluster. Iteratively split clusters until each data point is in its own cluster or a stopping criterion is met.

Dendrogram:

- A **tree-like diagram** used to represent the sequence of merges or splits.
- The **height** of each node in the dendrogram indicates the **distance** (or dissimilarity) at which clusters are joined or divided.
- To **decide the number of clusters**, draw a **horizontal cut** at the desired height.

Linkage Criteria (to compute inter-cluster distance):

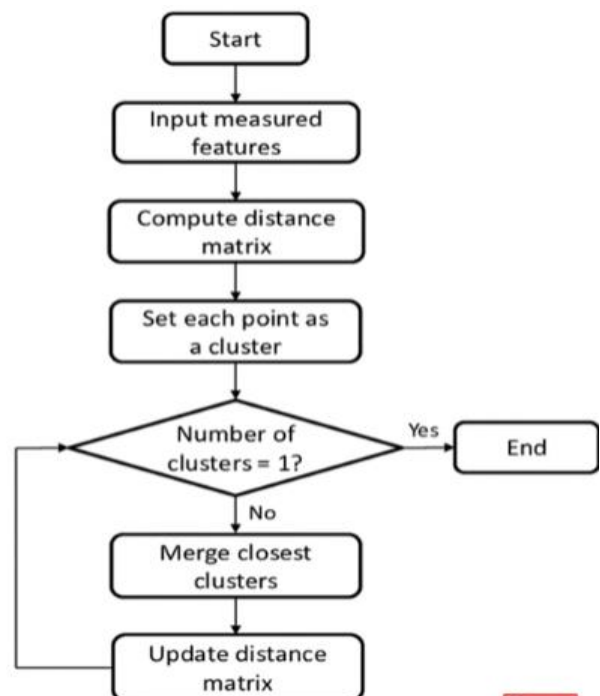
Linkage Type	Description
Single Linkage	Minimum distance between any two points (nearest neighbor)
Complete Linkage	Maximum distance between any two points (farthest neighbor)
Average Linkage	Average distance between all pairs of points from two clusters
Centroid Linkage	Distance between centroids of clusters
Ward's Method	Minimizes the total within-cluster variance

Advantages:

- No need to specify number of clusters in advance.
- Can produce meaningful hierarchy and dendrogram visualization.
- Flexible linkage methods.

Disadvantages:

- **Computationally expensive** for large datasets (time complexity: $O(n^3)$ and space $O(n^2)$).
- Once a merge or split is done, it **cannot be undone**.
- Sensitive to noise and outliers.
- Dendrogram can become difficult to interpret for large data.



Applications:

- Gene expression analysis
- Taxonomy and biology (organism classification)
- Social network analysis
- Document or web page clustering

Single Linkage(min) Agglomerative Clustering

	18	22	25	27	42	43
18	0					
22	4	0				
25	7	3	0			
27	9	5	2	0		
42	24	20	17	15	0	
43	25	21	18	16	1	0

Min distance between pair of points

is 42 & 43.

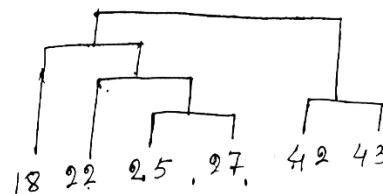
So we can merge them.

We consider Single linkage (min distance)

	18	22	25	27	(42, 43)
18	0				
22	4	0			
25	7	3	0		
27	9	5	2	0	
(42, 43)	24	20	17	15	0

Min distance is (25, 27)

Dendrogram ((42, 43), ((25, 27), 22), 18))



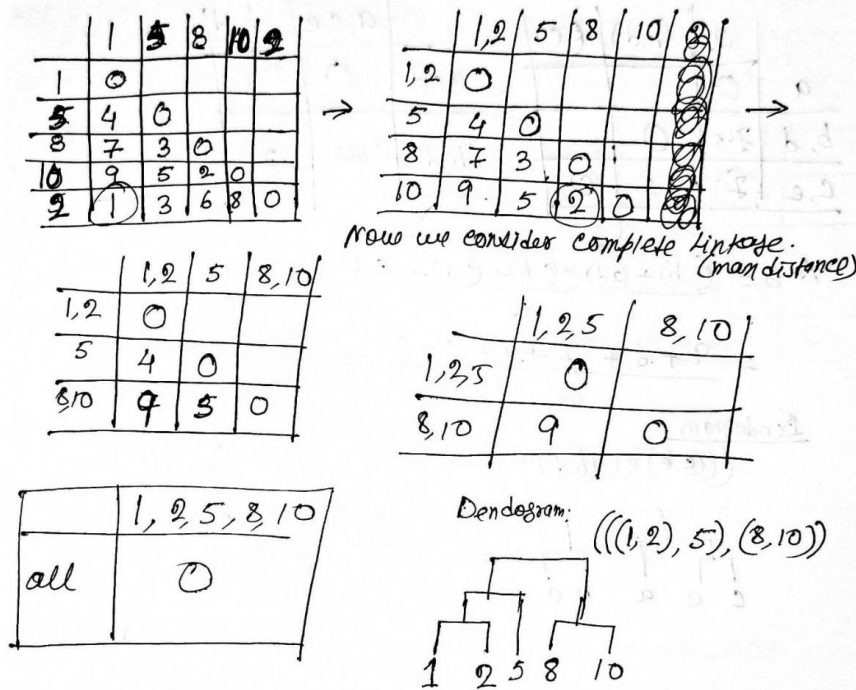
	18	22	(25, 27)	(42, 43)
18	0			
22	4	0		
(25, 27)	7	3	0	
(42, 43)	24	20	15	0

	18	(22, 25, 27)	(42, 43)
18	0		
(22, 25, 27)	4	0	
(42, 43)	24	15	0

	(18, 22, 25, 27)	(42, 43)
(18, 22, 25, 27)	0	
(42, 43)	15	0

	all
all	0

Complete Linkage (Max) Agglomerative Clustering



Average Linkage Agglomerative Clustering

